TD 1

1 Sac à dos

[ ]:
```
import numpy as np
```

2 Énoncé

Soit un sac-à-dos (sàd) de capacité volumique maximale de $V_{\max} = 10$ (que l'on ne précise pas ici). On souhaite y ranger trois objets de volumes respectifs $v_1 = 7$, $v_2 = 6$ et $v_3 = 4$. La question est délicate puisque:

$$\sum_{i=1}^{3} v_i > V_{\max}$$

Trouvez la valeur optimale de rangement, à savoir: *comment ranger le plus d'objets possible tout en laissant le moins de place libre dans le sàd?*

3 Analyse du problème

On va utiliser la progammation dynamique (PD) plutôt que la réccursivité afin d'*optimiser* l'algorithme en complexité temporelle.

La PD opère en deux phases : 1. Elle explore et enregistre *toutes* les solutions viables, 1. Elle rebrousse chemin en veillant à passer par le parcours *minimale*, c-à-d. en *minimisant* un critère donné.

Ce critère est la valeur optimale que l'on souhaite obtenir in fine:

$$\max \sum_{i=1}^{3} x_i v_i \leq V\max \quad \text{avec} \quad x_i \in \{0, 1\}$$

3.1 Décomposition du problème en suivant le critère

Supposons que la première phase soit effectuée correctement. On peut alors s'intéresser à la façon dont le chemin sera trouvé. À la fin de la première phase, tous les objets ont été testés. Ainsi pour le dernier objet de volume v_n soit il est présent dans le sàd soit il n'y est pas. En écrivant le critère sous la forme suivante:

$$OPT(n, V_{\max})$$

qui signifie que les n objets ont été testés et que OPT représente alors la valeur optimale. On constate aisément que:

$$\begin{cases} v_n \notin \text{Sàd} & \Longrightarrow OPT_{\notin}(n, V_{\max}) = OPT(n-1, V_{\max}) \\ v_n \in \text{Sàd} & \Longrightarrow OPT_{\in}(n, V_{\max}) = OPT(n-1, \boldsymbol{V_{\max} - v_n}) + \boldsymbol{v_n} \end{cases}$$

La valeur optiamle finale vaut donc:

$$OPT(n, V_{\max}) = \max\left(OPT_{\notin}(n, V_{\max}), OPT_{\in}(n, V_{\max})\right) \tag{1}$$

3.1.1 Remarque

Si l'on constate que $V_{\max} < v_n$ alors on en déduit que:

$$OPT(n, V_{\max}) = OPT(n-1, V_{\max}) \tag{2}$$

3.1.2 Question

Déployez l'arbre suivant le critère pour notre jeu d'essai.

3.1.3 Récurrence

Ce que l'on vient de faire pour le $n^{\text{ième}}$ objet se généralise à tous les objets:

$$OPT(i, V_{\max}) = \begin{cases} OPT(i-1, V_{\max}) & \text{si } V_{\max} < v_i \\ \max\left(OPT(i-1, V_{\max}), OPT(i-1, V_{\max} - v_i) + v_i\right) & \text{si } v_i \leq V_{\max} \end{cases} \qquad \forall i \in \{1, \ldots, n\}$$

3.1.4 Question

Muni des équations (1) et ((2), contruisez l'algorithme de la PD. Dans une première version, mémorisez uniquement les valeurs du critère OPT. Puis complétez votre algorithme pour qu'il enregistre également les chemins possibles dans l'espace de recherche.

[ ]:
```
# Données
Vmax = 10
vi = [0, 7, 6, 4] # plus volume nul
print( "Vmax =", Vmax, "vi =", vi )

# Intialisation à Nan
CHM = # TODO
OPT = # TODO
OPT[0,:] = # TODO
print( 'OPT=',OPT )

# Algo
for i in range(1,len(vi)):
    print( 'Object N°',i )
    for V in range(0,Vmax+1):
      """   TODO   """
    print( '\nOPT =\n', OPT, '\nCHM =\n', CHM )
```

4 Sac à dos avec utilités

On reprend les mêmes données auxquelles on ajoute des valeurs d'*utilité* à chaque objet:

$$\begin{array}{lll} v_1 = 7 & v_2 = 6 & v_3 = 4 \\ w_1 = 11 & w_2 = 8 & w_3 = 5 \end{array}$$

L'analyse précédente s'en trouve légèrement changée. Le critère porte sur la somme des utilités *contrainte* par le volume occupé dans le sac dos:

$$\max \left(\sum_{i=1}^{n} x_i w_i \right)_{\left| \sum_{i=1}^{n} x_i v_i \leq V_{\max} \right.} \qquad \text{avec } x_i \in \{0,1\}$$

Le critère maintenant manage à la fois le volume occupé dans le Sàd **et** la somme des utilités; *ce dernier point devient le critère effectif.*

$$\begin{cases} v_n \notin \text{Sàd} & \Longrightarrow \; OPT_{\notin}(n, V_{\max}) = OPT(n-1, V_{\max}) \\ v_n \in \text{Sàd} & \Longrightarrow \; OPT_{\in}(n, V_{\max}) = OPT(n-1, \boldsymbol{V_{\max} - v_n}) + \boldsymbol{w_n} \end{cases} \tag{3}$$

4.0.1 Questions

1. Écrivez la récurrence généralisant l'équation (3),
2. Déduisez en l'algorithme de la PD.

[ ]:
```
# Données
Vmax = 10
vi = [0, 7, 6,4] # plus volume nul
wi = [0, 11, 8, 5] # plus pondération nulle
print( "Vmax =", Vmax, "vi = ", vi )

# Intialisation
OPT = # TODO
OPT[0,:] = # TODO
print("OPT =\n", OPT)

# Algo
for i in range(1,len(vi)):
    print( '\n', i )
    for V in range(0,Vmax+1):
      """   TODO   """
    print(OPT)
```

5 Sac à dos avec espace d'états

5.1 Définitions

1. Un **état** est un couple $[u, v]$ où u est l'utilité et v le volume *occupé* par les objets rangés *jusque là*;
2. À l'étape i le **sous-espace** E_i contient les n_i états possibles.

5.1.1 Proposition

Soient les deux fonctions de projection des états, suivant les utilités et suivant les volumes:

$$U(E_i, v) = \{u_{i,j} : \forall [u_{i,j}, v] \in E_i\} \tag{1}$$
$$V(E_i) = \{v : \forall [u, v] \in E_i\} \tag{2}$$

On alors:

$$E_0 = \{\} \tag{3}$$
$$E'_i = E_{i-1} \bigcup \left\{ \left([u_{i-1,j}, v_{i-1,j}] + [w_i, v_i] \right) \ : \ \forall j \in \{1, \ldots, n_{i-1}\} \ v_{i-1,j} + v_i \leq V_{\max} \right\} \tag{4}$$
$$E_i = \left\{ [u, v] \ \in \ E'_i \ : \ u = \max \left(U \left(E'_i, \ v \right) \right), \ \forall v \in V \left(E'_i \right) \right\} \tag{5}$$

car on veut ne conserver que les sommets *d'utilité maximale* pour chaque valeur (*distincte*) de volume.

5.1.2 Question

Modifiez l'algorithme de la PD précédent pour n'enregistrer que les états significatifs.

[ ]:
```
# Données
Vmax = 10
vi = [0, 7, 6,4] # plus volume nul
wi = [0, 11, 8, 5] # plus pondération nulle
print( "Vmax =", Vmax, "vi =", vi )

# Intialisation
E = # TODO
E[0] = # TODO
print( "E0 =", E )

# Algo
for i in range(1,len(vi)):
    print( '\n', i )
    E[i] = []
    for [u,v] in E[i-1]:
      """   TODO   """
    print( "i =", i, "Ei =", E[i] )
```

Exo en cours :

Entrées : n tâches, p_j la durée opératoire de la tâche j
1 machine
$[t_1, t_2]$ l'intervalle d'indisponibilité

Objectif : Minimiser le makespan (C_j)



→ Maximiser la durée opératoire avant t_1 pour minimiser après t_2.
On peut appliquer le pb du sac à dos. On a :
- k objets
- w_i : utilité de l'objet i
- v_i : volume de l'objet i
- V : volume total.

Il faut donc choisir un sous-ensemble d'objets à placer avant t_1.

Ainsi :
* $V_m = t_1$.
* $w_i = p_j$
* $v_i = p_j$
* $k = n$.

Le pb peut se résumer avec : $opt(n, V_m)$ (nb objs, vol. tot.)

On prend : si $v - v_i \geq 0$, $opt(i, v_d) = opt(i-1, v_d - v_i) + w_i$

On ne prend pas : $opt(i, v_d) = opt(i-1, v_d)$

si $v - v_i \geq 0$.

→ Ça veut dire : $opt(i, v_d) = \max\{ opt(i-1, v_d), opt(i-1, v_d - v_i) + v_i \}$

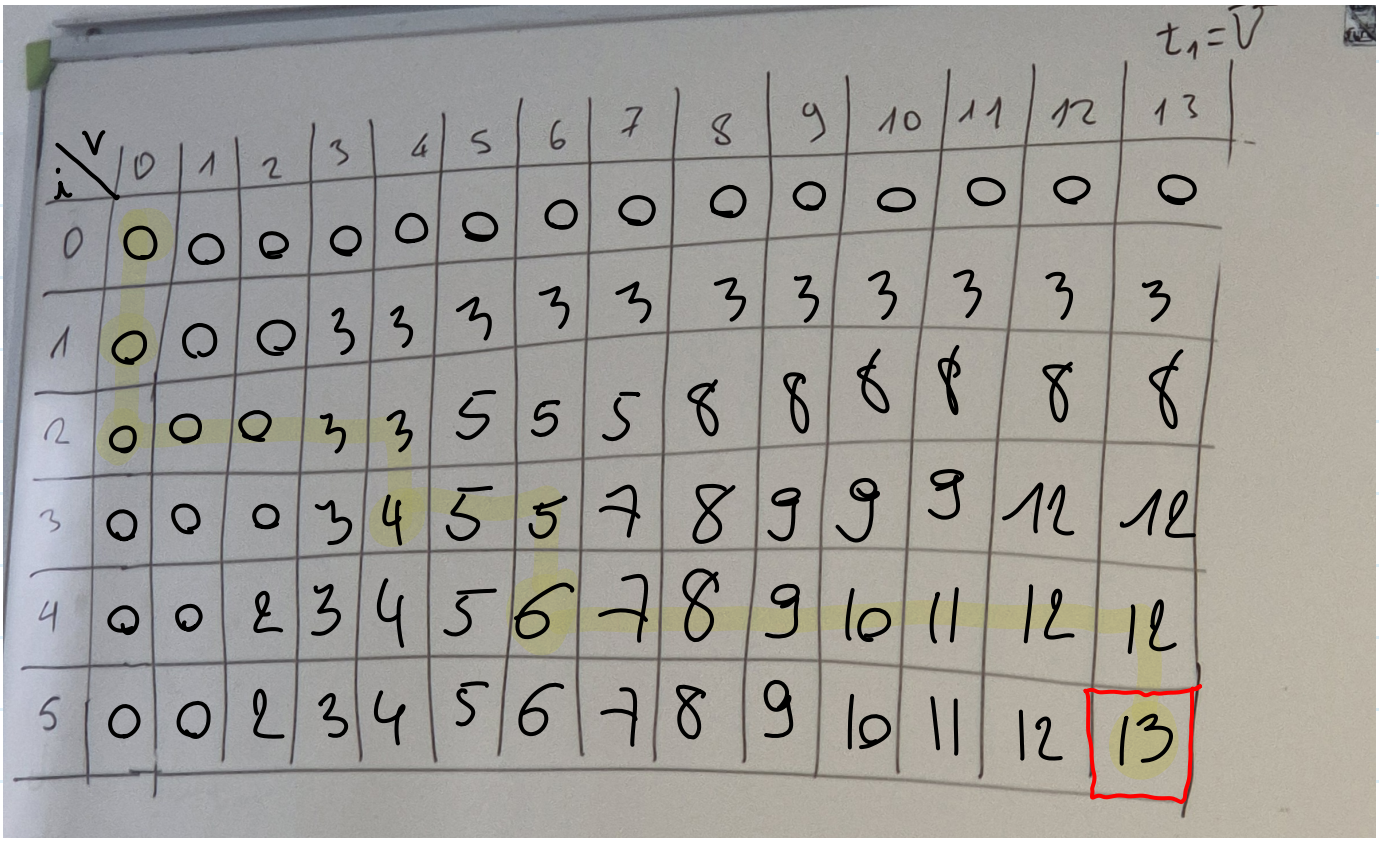
⚠ ne pas oublier les cas d'arrêt.

$$\begin{cases} opt(i, 0) = 0 \quad \forall i \leq k \\ opt(0, v_d) = 0 \quad \forall v_d \leq v_m. \end{cases}$$

On suppose les tâches :

j	1	2	3	4	5.	et	$t_1 = 13$
p_j	3	5	4	2	7.		$t_2 = 15$

Pour régler le pb du sac à dos, soit les tâches avant t_1 :



Ordre : $k \Leftarrow k-1$, $v \Leftarrow v - v_i$ donc de haut en bas, gauche à droite

Si on a ↑ : on ne prend pas. $v \to v$

$\leftrightarrow v_i$: on prend! $v \to v - v_i$.

Donc: (3,4,5) avant t_1 et (1,2) après. Ici l'ordre n'importe pas donc on peut les permuter.



* $C_{max} = 23$.

On a $(n+1)(V+1)$ cases, donc $O(NV)$ comparaisons.

Une instance pour le sac a une taille de:

$$|I|_2 = \log_2(N) + \log_2(V) + \sum_{i=1}^{N} \left(\log_2(v_i) + \log_2(w_i)\right)$$

$v_i \leqslant v_b$ et $w_i \leqslant w_b$.

$$|I|_2 \leqslant \log_2(N) + \log_2(V) + N\log_2(v_b) + N\log_2(w_b)$$

On peut dire $|I|_2 \simeq \log_2(V) \Rightarrow 2^{|I|_2} = 2^{\log_2(V)}$

$\Rightarrow V = 2^{|I|_2}$

Donc: $O(NV) \Rightarrow O(N 2^{|I|_2})$

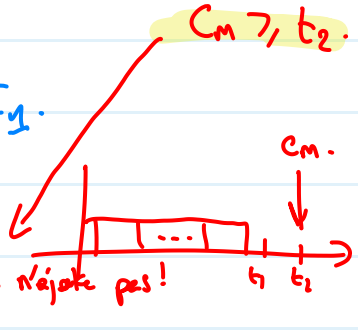
En unaire: $|I|_1 = \sum v_i + \sum w_i + V + N \to |I|_1 = O(V)$

Sans le sàd : Définir une formule sans regarder à dos.

On prend $opt(i, t)$ avec i la tâche $\&$ t la capacité à gauche de t_1.

On fait un appel comme $opt(n, t_1)$.

Le choix : mettre à gauche ou à droite de t_1.



À gauche : $opt(i,t) = opt(i-1, \underbrace{t-p_i}_{\geq 0})$

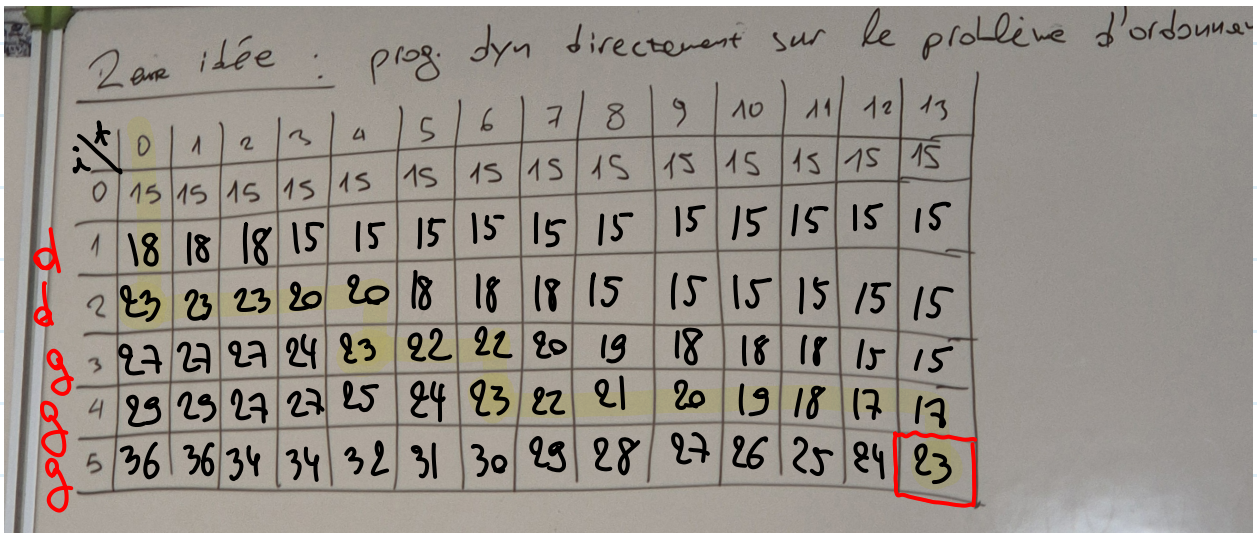
À droite : $opt(i,t) = opt(i-1,t) + p_i$ ici on rajoute car après t_2.

$$\Rightarrow opt(i,t) = \min \{ opt(i-1, t-p_i), opt(i-1,t) + p_i \}$$

Pour régler le pb du t_2 à ajter :

$$opt(0,t) = t_2. \qquad \forall t \in [\![0, t_1]\!]$$

Le tableau est $i \backslash t$



↰ : gauche. ↑ $+p_j$: droite.

Donc : (3, 4, 5) à gauche & 1, 2 à droite. 1 solut°.

Avec l'espace d'états : À l'itérat° i, on applique la transfo aux états précédents.

→ $[t, c]$: solut° partielle pour les i premières tâches où :
t : somme des durées avant t_1.
c : makespan, $c \geq t_2$.

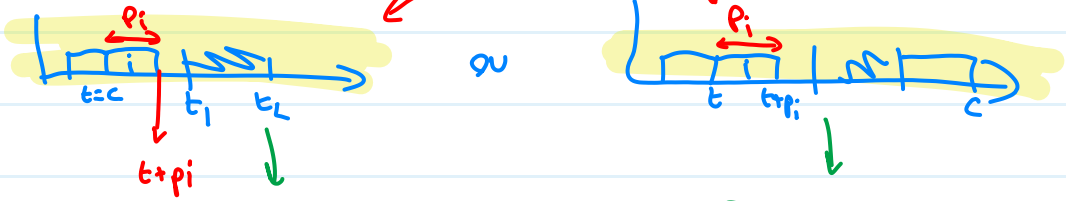
– E_i : Ens. d'états produits à l'itérat° i.

Règle d'évolut° : $f : [t, c]$:

À gauche :

$[t+p_i, \max\{c, t+p_i\}]$ si i à gauche.

si $t+p_i \leq t_1$.

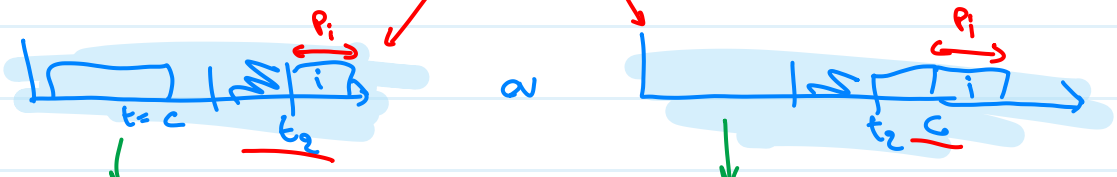


Aucune tâche à droite.

Déjà une tâche à droite.

À droite :

$[t, \max\{t_2, c\}+p_i]$ si i à droite.



1re tâche à droite

2e tâche à droite

$\Rightarrow$ On peut remplacer $\max\{t_2, C_j + p_i$ peut $\hat{\imath}$ remplacé par $t_2 + \sum_{p=1}^{i} p_i - t$.